



重慶大學
CHONGQING UNIVERSITY

研究生课程考查

科 目：智能控制理论及应用 教 师：*****

姓 名：***** 学 号：*****

专 业：电子信息 类 别：专业

上课时间：2026 年 3 月至 2026 年 6 月

考 生 成 绩：

平时成绩	卷面成绩	课程综合成绩

阅卷评语：

阅卷教师 (签名)

重庆大学研究生院制

目录

第一部分：文献综述报告	1
1 什么是玻尔兹曼机与受限玻尔兹曼机	1
1.1 发展历史	1
1.2 基本概念	1
2 基本思想和原理	2
2.1 核心原理: 能量函数与概率模型	2
2.2 RBM 推理: 正反方向的” 梦想” 过程	3
2.3 学习算法: 对比散度 (CD-k)	3
3 应用实例与仿真演示 (算法和仿真)	4
4 最新研究进展	6
4.1 最新研究进展: 从 RBM 到深度与变分	6
4.2 现代应用中的演变: 量子计算与边缘计算	6
5 评价: 优缺点与历史地位	7
6 主要参考文献	7
 第二部分：仿真实验报告	 9
1 引言	9
1.1 概述	9
1.2 被控对象分析	9
2 传统 PID 设计方案	10
2.1 设计思想	10
2.2 控制器设计	10
2.3 参数整定	11
2.3.1 IMC-PID 整定方法	11
2.3.2 针对两种工况的参数确定	11
2.4 伺服控制仿真	12
2.5 定值控制仿真	13

3	智能控制方案	14
3.1	模糊控制原理	14
3.2	模糊 PID 控制器设计	14
3.2.1	控制器结构	14
3.2.2	隶属度函数设计	14
3.2.3	模糊规则库	15
3.2.4	推理与解模糊	15
3.2.5	参数配置	16
3.3	仿真结果	16
4	两方案对比	18
4.1	性能指标对比	18
4.2	智能控制方案效果分析	18
4.2.1	IMC 基准已接近最优	18
4.2.2	稳定性约束限制了调整幅度	19
4.2.3	大延迟对模糊推理的限制	19
4.2.4	随机扰动工况的特殊性	19
4.3	模糊 PID 的合理应用场景	19
5	心得体会	20
6	参考文献	21
7	附录	22
7.1	仿真代码: 控制器模块	22
7.2	仿真代码: 仿真模块	31



重庆大学
CHONGQING UNIVERSITY

玻尔兹曼机与受限玻尔兹曼机综述

姓名: 林沁朋
学号: *****
日期: 2026年4月18日



重庆大学
CHONGQING UNIVERSITY

目录

- 1. 什么是玻尔兹曼机与受限玻尔兹曼机
 - 1.1 发展历史
 - 1.2 基本概念
- 2. 基本思想和原理
 - 2.1 核心原理：能量函数与概率模型
 - 2.2 RBM推理：正反向的“梦想”过程
 - 2.3 学习算法：对比散度 (CD-k)
- 3. 应用实例与仿真演示（算法和仿真）
- 4. 最新研究进展
 - 4.1 最新研究进展：从RBM到深度与变分
 - 4.2 现代应用中的演变：量子计算与边缘计算
- 5. 评价：优缺点与历史地位
- 6. 主要参考文献



重庆大学
CHONGQING UNIVERSITY

- 我们决计没有办法用我们的一些不完备的实验来考查这组或那组物理定律是否可以验证到最后一位小数。但是，牛顿的观点就迫使人们把物理学陈述得并且用公式表示成好象它真的是受着这类定律支配的样子。现在，这种观点在物理学中已经不居统治地位了，而对推翻这种观点出力最多的人就是德国的玻耳兹曼 (Boltzmann) 和美国的吉布斯(Gibbs)。
- 这两位物理学家都是彻底地应用了一个激动人心的新观念的。他们在物理学中所大量引进的统计学
- 《人有人的用处：信息论与社会》维纳



重庆大学
CHONGQING UNIVERSITY

1. 什么是玻尔兹曼机与受限玻尔兹曼机

- 1.1 发展历史
- 1.2 基本概念



重庆大学
CHONGQING UNIVERSITY

1.1 发展历史

1985年：玻尔兹曼机 (BM) 的诞生
(正式提出玻尔兹曼机及其学习算法，解决了多层神经网络内部表征的学习问题 Ackley, D. H., Hinton, G. E., & Sejnowski, T. J. (1985). A learning algorithm for Boltzmann machines. *Cognitive Science*, 9(1), 147-169.)

1986年：受限玻尔兹曼机 (RBM) 结构的提出
(首次提出了“Harmonium” (簧风琴)，即后来的受限玻尔兹曼机) Smolensky, P. (1986). Information processing in dynamical systems: Foundations of harmony theory. In D. E. Rumelhart & J. L. McClelland (Eds.), *Parallel Distributed Processing* (Vol. 1, Chapter 6).

2002年：对比散度 (CD) 算法的突破
(CD算法极大提升了训练效率，是RBM走向实用的关键) Hinton, G. E. (2002). Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8), 1771-1800.

2006年：深度信念网络 (DBN) 与深度学习复兴
(奠定了“逐层无监督预训练+反向传播微调”的深度学习范式，标志着深度学习时代的开启) Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504-507.

2024年：诺贝尔物理学奖加冕
Hinton, G. E. (2025). Boltzmann machines. *Reviews of Modern Physics*, 97, 030502.



重庆大学
CHONGQING UNIVERSITY

1.2.1 基本概念：什么是玻尔兹曼机 (BM)

- 本质：一种基于能量的随机递归神经网络。
- 结构图：展示全连接图（可见单元+隐藏单元，互相之间都有双向连接）。（图论：完全图）
- 核心思想：模拟物理系统的热平衡状态，通过模拟退火算法寻找全局能量最低点（最优解）。
- 痛点：训练极慢，计算复杂度高（无法实用化）。



概率无向图模型，用能量衡量整个模型的稳定性

特点1：每个随机变量都是二值的{0, 1}

特点2：所有节点都是全连接的(图论：完全图)

特点3：两个节点之间的相互影响是对称的

$$E(X=x) = -(\sum_{i,j} w_{ij}x_i x_j + \sum_i b_i x_i)$$

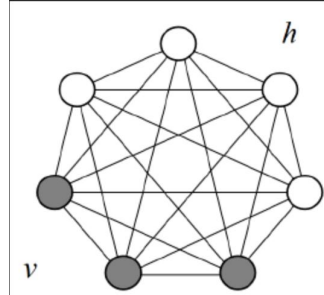


图 1: Boltzmann machine 模型的概率图



1.2.2 基本概念：受限玻尔兹曼机(RBM)

• **改进本质：**BM的二部图特化版本。

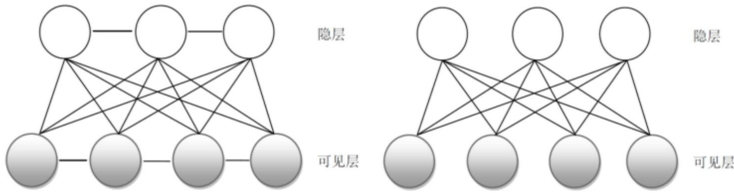
• **结构图：**展示二分图（层内无连接，只有层间全连接 $V \leftrightarrow H$ ）。（图论：完全二部图）

• **核心优势：**

1. 可见层给定下，隐层神经元条件独立。
2. 训练效率大幅提升（引入**对比散度 CD 算法**）。
- **功能定位：**特征提取器、降维工具、生成模型的基块。



BM与RBM结构对比



(a) BM模型的结构.

(b) RBM模型的结构.

$$E(x, h) = -h^T W x - c^T x - b^T h$$

$$-\frac{1}{2} x^T V x - \frac{1}{2} h^T U h$$

知乎 @华年ss



2.玻尔兹曼机与受限玻尔兹曼机的基本思想和原理

- 2.1 能量模型与概率分布
- 2.2 RBM 的推理过程
- 2.3 学习算法：对比散度(CD)



2.1 核心原理：能量函数与概率模型

• **公式：** RBM 能量函数公式

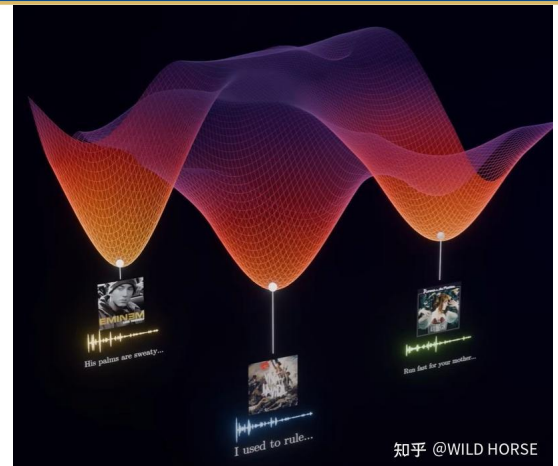
$$E(v, h) = -a^T v - b^T h - v^T W h$$

• **核心逻辑：** 能量越低，系统越稳定，概率越大。
(可以联想到李雅普诺夫稳定性理论)

• **联合概率公式** $P(v, h) \propto e^{-E(v, h)}$



玻尔兹曼机物理直觉理解



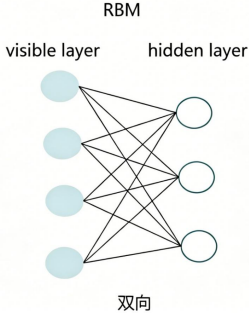
知乎 @WILD HORSE



2.2 RBM 推理：正反向的“梦想”过程

受限玻尔兹曼机 (Restricted Boltzmann Machine)

层内的神经元没有连接关系



豆包AI生成

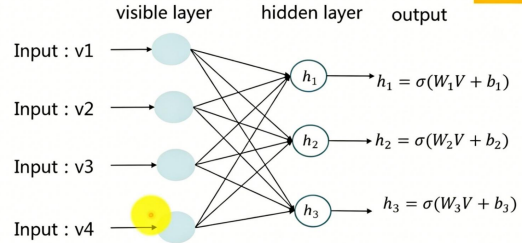


2.2 RBM 推理：正反向的“梦想”过程

受限玻尔兹曼机传播过程 – 正向

$$p(h|v)$$

$$p(h_j|v) = \sigma(\sum_i W_{ij}v_i + b_j)$$



豆包AI生成

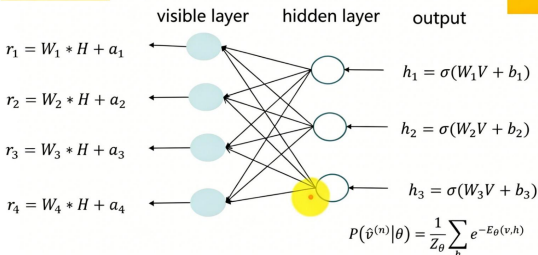


2.2 RBM 推理：正反向的“梦想”过程

受限玻尔兹曼机传播过程 – 反向

$$p(v|h)$$

$$p(v_i|h) = \sigma(\sum_j W_{ij}h_j + a_i)$$



训练过程：使预测值R逐渐逼近输入X。

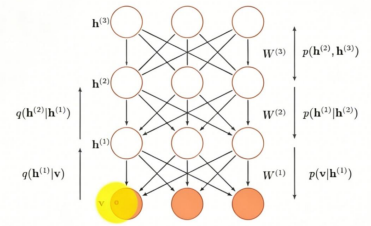
$$E_{\theta}(v, h) = -\sum_{i=1}^n \sum_{j=1}^m w_{ij} v_i h_j - \sum_{i=1}^n a_i v_i - \sum_{j=1}^m b_j h_j$$

豆包AI生成



深度置信网络 (Deep Belief Networks)

最顶层两层的连接是无向的



训练法则：逐层进行，每一层都用用户数向量来推断隐层，再把这个隐层作为数据向量来推断下一个隐层。



2.3 学习算法：对比散度 (CD-k)

• 对比：精确计算期望 (难) vs Gibbs 采样近似。

• CD-1 流程：

- 1.. 输入数据 $v^{(0)} \rightarrow$ 计算隐层 $h^{(0)}$
- 2.. 重构可见层 $v^{(1)}$ 。
- 3.. 计算重构后的隐层 $h^{(1)}$ 。
- 4.. 更新规则： $\Delta W \approx \langle v h^T \rangle_{data} - \langle v h^T \rangle_{recon}$ 。

• 比喻：让学生记住真实图片的样子，忘记自己乱画的涂鸦的样子。



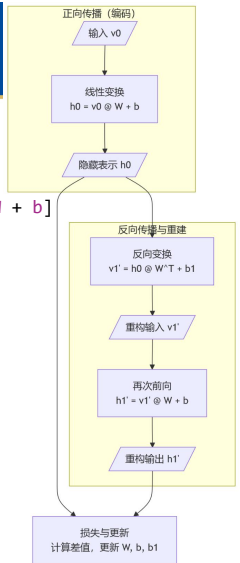
CD_1(对比散度)伪代码

```
# 假设已经有一个训练样本 v0 (形状为 784 的一维向量)
# 已有参数: 权重 W, 可见偏置 a, 隐偏置 b
def CD_1(v0, W, a, b, lr=0.1):
    # 1. 正向: 由真数据 v0 得到隐层概率 h0_prob, 并采样
    h0_prob = sigmoid(b + v0 @ W)
    h0 = (h0_prob > random.uniform(0,1)).astype(float) # 二值采样
    # 2. 反向: 重构可见层 v1_prob, 并采样
    v1_prob = sigmoid(a + h0 @ W.T)
    v1 = (v1_prob > random.uniform(0,1)).astype(float)
    # 3. 再次正向: 由重构数据 v1 得到隐层 h1_prob (CD-1 中通常不用再采样 h1, 直接用概率)
    h1_prob = sigmoid(b + v1 @ W)
    # 4. 对比散度: 计算正相位 - 负相位, 然后更新参数
    pos_grad = v0.reshape(-1,1) @ h0.reshape(1,-1) # 外积
    neg_grad = v1.reshape(-1,1) @ h1_prob.reshape(1,-1)
    # 更新权重
    W += lr * (pos_grad - neg_grad)
    a += lr * (v0 - v1) # 可见偏置更新
    b += lr * (h0 - h1_prob) # 隐偏置更新
    return W, a, b
```

CD - 1数据流图

% CD - 1 算法数据流图
flowchart TB

```
subgraph forward ["正向传播 (编码)"]  
    A[/输入 v0/] --> C[线性变换  
h0 = v0 @ W + b]  
    C --> B[/隐藏表示 h0/]  
end  
  
subgraph backward ["反向传播与重建"]  
    D[反向变换  
v1' = h0 @ W^T + b1]  
    D --> D1[/重构输入 v1'/]  
    D1 --> C1[再次前向  
h1' = v1' @ W + b]  
    C1 --> B1[/重构输出 h1'/]  
end  
  
B --> D  
B --> E  
B1 --> E  
E["损失与更新  
计算差值, 更新 W, b, b1"]
```



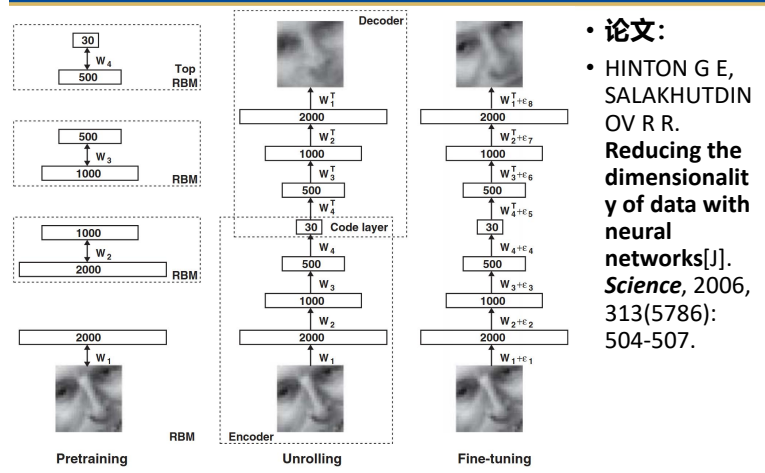
应用实例与仿真演示

3. 应用实例与仿真演示

- 3.1 手写数字特征提取与重构
- 3.2 协同过滤推荐系统



3.1 仿真案例一：MNIST 数字生成与降噪



• 论文:
• HINTON G E, SALAKHUTDIN OV R R. Reducing the dimensionality of data with neural networks[J]. Science, 2006, 313(5786): 504-507.

Reducing the Dimensionality of Data with Neural Networks

G. E. Hinton* and R. R. Salakhutdinov

Fig. 2. (A) Top to bottom: Random samples of curves from the test data set; reconstructions produced by the six-dimensional deep autoencoder; reconstructions by "logistic PCA" (6) using six components; reconstructions by logistic PCA and standard PCA using 18 components. The average squared error per image for the last four rows is 1.44, 7.64, 2.45, 5.90. (B) Top to bottom: A random test image from each class; reconstructions by the 30-dimensional autoencoder; reconstructions by 30-dimensional logistic PCA and standard PCA. The average squared errors for the last three rows are 3.00, 8.01, and 13.87. (C) Top to bottom: Random samples from the test data set; reconstructions by the 30-dimensional autoencoder; reconstructions by 30-dimensional PCA. The average squared errors are 126 and 135.

Figure 2 shows three rows of handwritten digits. Row A shows random samples of curves, reconstructions by a six-dimensional deep autoencoder, and reconstructions by logistic PCA and standard PCA. Row B shows a random test image from each class, reconstructions by a 30-dimensional autoencoder, and reconstructions by 30-dimensional logistic PCA and standard PCA. Row C shows random samples from the test data set, reconstructions by a 30-dimensional autoencoder, and reconstructions by 30-dimensional PCA.

论文意义

- 提出了逐层+预训练+微调的模型学习范式，解决了多层网络在训练时的容易陷入局部最优化和梯度消失问题。
- 自此，深度学习复兴。

层 (2-4)。对于较大的初始权重，自动编码器通常会找到较差的局部最小值；对于较小的初始权重，早期层中的梯度很小，这使得训练具有许多隐藏层的自动编码器变得不可行。如果初始权重接近一个好的解决方案，梯度下降效果很好，但是找到这样的初始权重需要一种非常不同类型的算法，它一次学习一层特征。我们为二进制数据引入了这个8预训练过程，将其推广到实值数据，并表明它适用于各种数据集。一个集成e。我们为二进制数据引入了这个8预训练过程，将其推广到实值数据，并表明它适用于各种数据集。

对于较大的初始权重，自动编码器通常会找到较差的局部最小值；对于较小的初始权重，早期层中的梯度很小，这使得训练具有许多隐藏层的自动编码器变得不可行。如果初始权重接近一个好的解决方案，梯度下降效果很好，但是找到这样的初始权重需要一种非常不同类型的算法，它一次学习一层特征。我们为二进制数据引入了这个8预训练过程，将其推广到实值数据，并表明它适用于各种数据集。

论文意义

An ensemble of binary vectors (e.g., images) can be modeled using a two-layer network called a "restricted Boltzmann machine" (RBM) (5, 6) in which stochastic, binary pixels are connected to stochastic, binary feature detectors using symmetrically weighted connections. The pixels correspond to "visible" units of the RBM because their states are observed; the feature detectors correspond to "hidden" units. A joint configuration (v, h) of the visible and hidden units has an energy (7) given by

$$E(v, h) = - \sum_{i \in \text{pixels}} b_i v_i - \sum_{j \in \text{features}} b_j h_j - \sum_{i,j} v_i h_j w_{ij} \quad (1)$$

where v_i and h_j are the binary states of pixel i and feature j , b_i and b_j are their biases, and w_{ij} is the weight between them. The network assigns a probability to every possible image via this energy function, as explained in (8). The probability of a training image can be raised by

- 极强的可解释性：
- 像素对应可见层
- 特征对应隐藏层

ChatGPT?

- ChatGPT全名: Chat Generative Pre-trained Transformer
- 自编码器和 编解码器

自编码器类似于要求Y=X的
编码器 + 解码器 (编解码器)

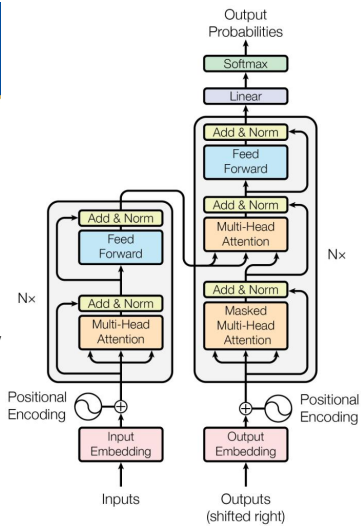
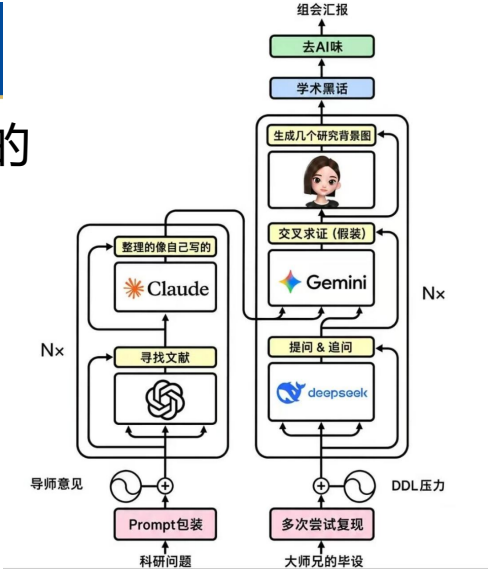


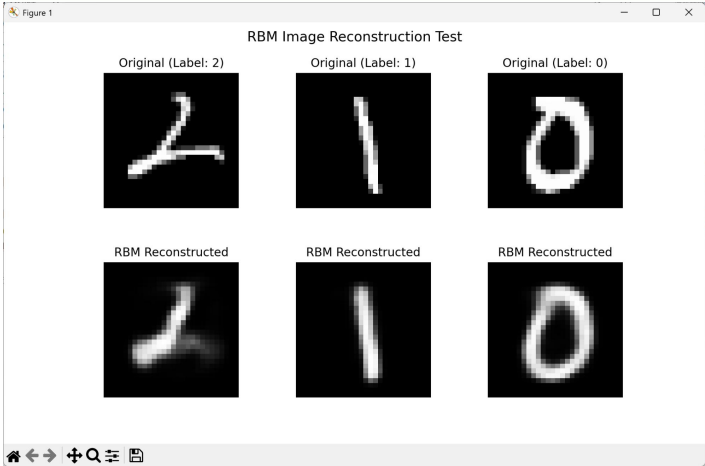
Figure 1: The Transformer - model architecture.

我搞科研时的 belike (bushi)

我搞科研时的 belike (bushi)



3.1 仿真案例一: MNIST 数字生成与降噪



3.2 仿真案例二: 电影评分预测 (协同过滤)

• 论文: SALAKHUTDINOV R, MNIH A, HINTON G. **Restricted boltzmann machines for collaborative filtering** [C]//Proceedings of the 24th international conference on Machine learning. Corvallis Oregon USA: ACM, 2007: 791-798. (当年打败了网飞的推荐系统!)

Abstract

Most of the existing approaches to collaborative filtering cannot handle very large data sets. In this paper we show how a class of two-layer undirected graphical models, called Restricted Boltzmann Machines (RBM's), can be used to model tabular data, such as user's ratings of movies. We present efficient learning and inference procedures for this class of models and demonstrate that RBM's can be successfully applied to the Netflix data set, containing over 100 million user/movie ratings. We also show that RBM's slightly outperform carefully-tuned SVD models. When the predictions of multiple RBM models and multiple SVD models are linearly combined, we achieve an error rate that is well over 6% better than the score of Netflix's own system.

3.2 仿真案例二: 电影评分预测 (协同过滤)

=====

🏠 【课堂演示】用户评分填补与电影推荐

=====

👤 演示用户 ID: 1

📺 用户【已看过】的电影及真实评分:

- Toy Story (1995): ★★★★★ (5星)
- GoldenEye (1995): ★★★★★ (3星)
- Four Rooms (1995): ★★★★★ (4星)
- Get Shorty (1995): ★★★★★ (3星)
- Copycat (1995): ★★★★★ (3星)

📺 RBM 填补【未看过】的电影预测评分 (Top 5 推荐):

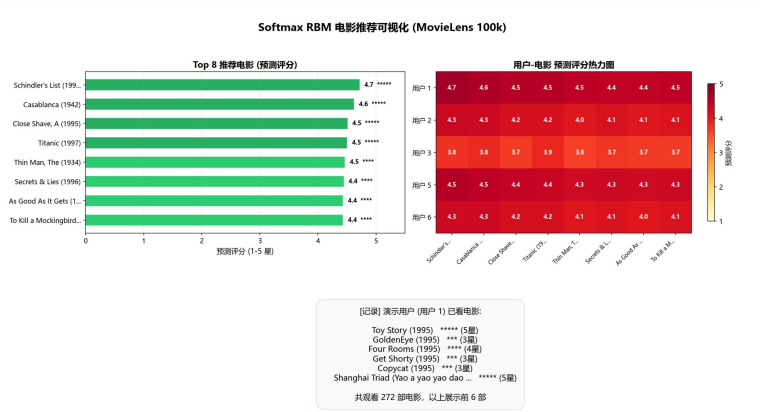
- Schindler's List (1993): 预测 4.67 星 ★★★★★
- Casablanca (1942): 预测 4.64 星 ★★★★★
- Close Shave, A (1995): 预测 4.55 星 ★★★★★
- Titanic (1997): 预测 4.52 星 ★★★★★
- Thin Man, The (1934): 预测 4.46 星 ★★★★★

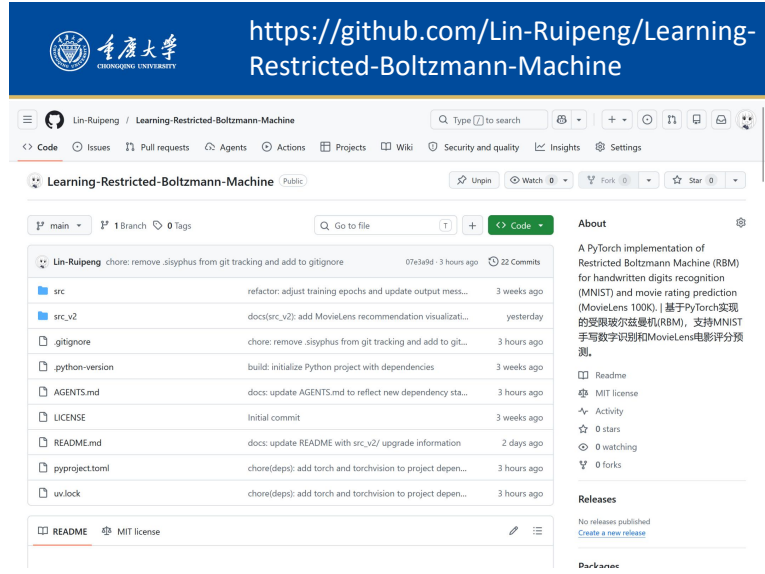
📌 注意: 以上预测评分仅供参考, 实际评分可能因用户喜好不同而有所差异。

→ Project git:(main) □

• 解释: 用户ID 1:
• 他看过的电影和评分:
《玩具总动员》给了5星,
《黄金眼》给了3星等等。
• RBM给他推荐的Top 5电影:
《辛德勒的名单》预测4.67星,
《卡萨布兰卡》4.64星,
还有《泰坦尼克号》等。

3.2 仿真案例二: 电影评分预测 (协同过滤)





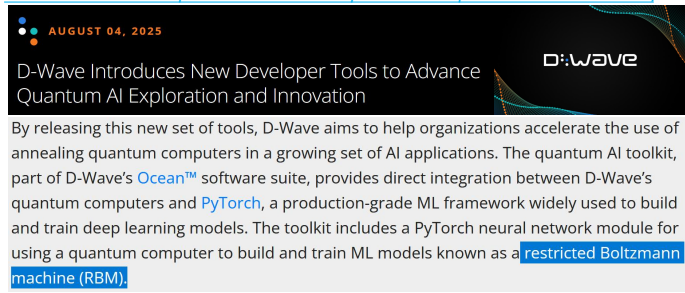
4.1 从RBM到深度生成模型

- 深度信念网络 (Deep Belief Network, DBN) 由Geoffrey Hinton于2006年提出，是深度学习历史上第一个真正意义上获得成功的深度生成模型
- 深度玻尔兹曼机 (Deep Boltzmann Machine, DBM) 是RBM家族的另一个重要分支。它同样由多层RBM堆叠而成，但每一层之间都保持无向连接，是一个完全的无向概率图模型 (DBM正在经历“复兴”)：稀疏伊辛机 (Ising machine)
- 与变分自编码器 (VAE) 的联系：RBM和VAE虽然从不同的理论起点出发，但在概率建模的深层逻辑上有着紧密的联系。最核心的联系体现在：RBM可以被用作VAE隐变量的先验分布。

4.2 量子计算赋能RBM

- 稀疏伊辛机 (2024 Nature Electronics)：UCSB团队提出用概率计算机 (p-bit伊辛机) 来训练深度玻尔兹曼网络，每秒可产生约500亿次概率采样。他们发现，仅3万参数的稀疏DBM能达到325万参数全连接RBM的精度，采样速度比GPU/TPU快至少一个量级。
(链接<https://opus.ece.ucsb.edu/news/2024/shailas-work-deep-boltzmann-machines-out-nature-electronics> (Nature的子刊，影响因子超过40，在CR分区中排名前0.1%。))
- Shaila's work on Deep Boltzmann Machines out in Nature Electronics!
Shaila's paper on Sparse Ising Machines appears in Nature Electronics
June 17, 2024
- 1. We show that energy-based "deep" Boltzmann machines (BM), traditionally considered to be intractable, can be trained using dedicated probabilistic computers where up to ~50 billion probabilistic samples per second are taken.
- 2. We find that the deep but sparse BMs with only 30,000 parameters can reach beyond the capabilities of fully-connected Restricted BMs with 3,000,000 parameters, where sparse BMs can generate new images where RBMs fail.
- 3. We train the full MNIST, Fashion MNIST and CIFAR datasets which shows promising classification accuracy "out-of-the-box" without any algorithmic fine-tuning.
- We are very excited about the potential of p-bit based Ising Machines (via their digital as well as scaled analog realization) on generative, physics-based AI algorithms.

4.2 量子计算赋能RBM

- D-Wave量子退火：2025年8月，D-Wave发布了量子AI工具包，让开发者可以直接在PyTorch中调用量子计算机来训练RBM。量子退火的核心优势在于——它天然适合从玻尔兹曼分布中高效采样。（官网）
 - (链接<https://www.dwavequantum.com/company/newsroom/press-release/d-wave-introduces-new-developer-tools-to-advance-quantum-ai-exploration-and-innovation/>)
- 

4.3 RBM在边缘计算中的应用

- RBM-LSTM异常检测：2024年有研究提出将RBM与LSTM结合，用于物联网 (IoT) 网络中路由攻击的异常检测。RBM负责无监督特征提取，LSTM负责时序预测。
- 为什么RBM适合边缘端？RBM结构简单、参数量少、推理过程只有矩阵乘法+Sigmoid，对算力要求低，非常适合部署在资源受限的边缘设备上。
- 论文：RASHMI S, ANAND N, KUMAR S R, et al. Routing attack induced anomaly detection in IoT network using RBM-LSTM[J]. ICT Express, 2024, 10(3): 459-464.
The network of resource constraint devices, also known as the Low power and Lossy Networks (LLNs), constitutes the edge tire of the Internet of Things applications like smart homes, smart cities, and connected vehicles. The IPv6 Routing Protocol over Low power and lossy networks (RPL) ensures efficient routing in the edge tire of the IoT environment. However, RPL has inherent vulnerabilities that allow malicious insider entities to instigate several security attacks in the IoT network. As a result, the IoT networks suffer from resource depletion, performance degradation, and traffic disruption. Recent literature discusses several machine learning algorithms to detect one or more routing attacks. However, IoT infrastructures are expanding, and so are the attack surfaces. Therefore, it is essential to have a solution that can adapt to this change. This paper introduces a comprehensive framework to detect routing attacks within Low Power and Lossy Networks (LLNs). The proposed solution leverages deep learning by combining Restricted Boltzmann Machine (RBM) and Long Short-Term Memory (LSTM). The framework is trained on 11 network parameters to understand and predict normal network behavior. Anomalies, identified as deviations from the forecast trends, serve as indicators of potential routing attacks and thus address vulnerabilities in the RPL.



重庆大学
CHONGQING UNIVERSITY



5. 评价：优缺点与历史地位

- 5.1 RBM的优缺点
- 5.2 历史地位：从复兴到正名
- 5.3 总结和展望



重庆大学
CHONGQING UNIVERSITY

5.1 RBM的优缺点

优点	缺点
1. 理论优美，有明确的能量/概率解释	1. 对比散度算法有偏，训练仍较慢
2. 强大的无监督特征提取能力	2. 高维图像受限：处理高维图像不如 CNN + GAN 精细
3. 生成能力：生成模型（区别于单纯的分类器）	



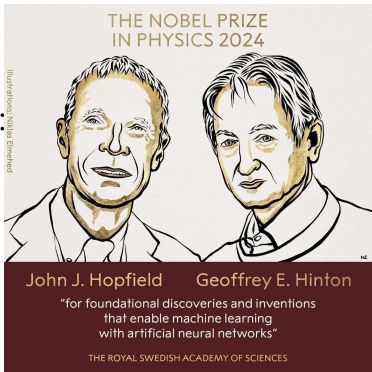
重庆大学
CHONGQING UNIVERSITY

5.2 历史地位：从复兴到正名



重庆大学
CHONGQING UNIVERSITY

- 2006年Science论文：把深度学习推回了舞台中央。
- 2007年Netflix协同过滤论文：RBM不仅在学术上有价值，在工业界也能打。
- 2024年诺贝尔物理学奖授予Hopfield和Hinton，颁奖词中明确提到了玻尔兹曼机作为获奖工作的核心贡献之一



- 5.3 总结和展望
- 一个“好模型”的评判标准不是它现在有多火，而是它的核心思想——基于能量的概率建模、无监督逐层预训练、用物理原理驱动学习——能不能持续启发后续的研究。
- 启示：不要只看当下最火的SOTA模型，理解经典模型背后的第一性原理，往往能让你在新问题面前找到独特的解决方案。



重庆大学
CHONGQING UNIVERSITY





重庆大学
CHONGQING UNIVERSITY

参考文献

6. 主要参考文献

标题	创建者	年	期刊标签
A learning algorithm for boltzmann machines	Ackley 等	1985	SSCI Q2 SCI升级版 心理学2区 SCIF 2.4 SCIF(S) 2.6 JCI 0.86
Training products of experts by minimizing contrastive ...	Hinton	2002	SCI Q3 SCI升级版 计算机科学4区 E检索 SCIF 2.1 SCIF(S) 4.8 JCI
Reducing the dimensionality of data with neural netwo...	Hinton...	2006	SCI Q1 SCI升级版 综合性期刊1区 SCIF 45.8 SCIF(S) 49.7 JCI 8.98
Restricted boltzmann machines for collaborative filtering	Salakh...	2007	
A practical guide to training restricted boltzmann mac...	Hinton	2012	
An introduction to restricted boltzmann machines	Hutchis...	2012	
玻尔兹曼机研究进展	刘建伟 等	2014	CQU 权威期刊 E检索 CCF T1
受限玻尔兹曼机的参数选择理论及应用	赵晓朝	2017	
受限玻尔兹曼机的改进及其应用	罗剑江	2018	
Restricted boltzmann machines: introduction and review	Montúfar	2018	
受限玻尔兹曼机研究综述	张健 等	2019	CQU 权威期刊 E检索 CCF T1
量子玻尔兹曼机的几何与对称性	朱海青	2021	
基于深度玻尔兹曼机的遥感影像变化检测方法	于杰 等	2023	
基于受限玻尔兹曼机的电商推荐算法研究	田璐	2024	
受限玻尔兹曼机及其度体研究综述	汪建龙 等	2024	CQU 重要期刊 E检索
Routing attack induced anomaly detection in IoT netw...	Rashmi ...	2024	SCI Q2 SCI升级版 计算机科学3区 SCIF 4.2 SCIF(S) 4.4 JCI 1.16
基于深度玻尔兹曼机的遥感影像变化检测方法	于杰	2025	

[1] ACKLEY D H, HINTON G E, SEJNOWSKI T J. A learning algorithm for boltzmann machines[J]. Cognitive Science, 1985, 9(1): 147-169.

[2] HINTON G E. Training products of experts by minimizing contrastive divergence[J]. Neural Computation, 2002, 14(8): 1771-1800.

[3] HINTON G E, SALAKHUTDINOV R R. Reducing the dimensionality of data with neural networks[J]. Science, 2006, 313(5786): 504-507.

[4] SALAKHUTDINOV R, MNIH A, HINTON G. Restricted boltzmann machines for collaborative filtering[C]//Proceedings of the 24th international conference on Machine learning. Corvallis Oregon USA: ACM, 2007: 791-798.

[5] HINTON G E. A practical guide to training restricted boltzmann machines[M]//Lecture notes in computer science: Vol. 7700. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012: 599-619.

[6] HUTCHISON D, KANADE T, KITTLER J, et al. An introduction to restricted boltzmann machines[M]//ALVAREZ L, MEJAIL M, GOMEZ L, et al. Lecture notes in computer science: Vol. 7441. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012: 14-36.

[7] 刘建伟, 刘媛, 罗雄麟. 玻尔兹曼机研究进展[J]. 计算机研究与发展, 2014, 51(1): 1-16.

[8] 赵晓朝. 玻尔兹曼机的参数选择理论及应用[D]. 天津: 天津大学, 2017.

[9] MONTÚFAR G. Restricted boltzmann machines: introduction and review[M]//AY N, GIBILISCO P, MATÚŠ F. Springer proceedings in mathematics & statistics: Vol. 252. Cham: Springer International Publishing, 2018: 75-115.

[10] 罗剑江. 受限玻尔兹曼机的改进及其应用[D]. 广州: 广东工业大学, 2018.

 重庆大学
CHONGQING UNIVERSITY

参考文献

[11] 张健, 丁世飞, 张楠, 等. 受限玻尔兹曼机研究综述[J]. 软件学报, 2019, 30(7): 2073-2090.
[12] 宋海青. 量子玻尔兹曼机的几何与对称性[D]. 北京: 中国科学院大学 (中国科学院物理研究所), 2021.
[13] 于杰, 李大成, 和栋材, 等. 基于深度玻尔兹曼机的遥感影像变化检测方法[J]. 无线工程, 2023, 53(12): 2905-2913.
[14] 汪强龙, 高晓光, 吴必聪, 等. 受限玻尔兹曼机及其变体研究综述[J]. 系统工程与电子技术, 2024, 46(7): 2323-2345.
[15] RASHMI S, ANAND N, KUMAR S R, et al. Routing attack induced anomaly detection in IoT network using RBM-LSTM[J]. ICT Express, 2024, 10(3): 459-464.
[16] 田璐. 基于受限玻尔兹曼机的电商推荐算法研究[D]. 济南: 山东师范大学, 2024.
[17] 于杰. 基于深度玻尔兹曼机的遥感影像变化检测方法[D]. 太原: 太原理工大学, 2025.

 重庆大学
CHONGQING UNIVERSITY

Q&A 与致谢

• 感谢聆听，请老师和同学提问。

Connect with me: GitHub & QQ



Fox Lin (林睿朋)
Lin-Ruipeng · he/him
Engineering graduate student at CQU
focusing on Edge AI Deployment and HPC.

 重庆大学
CHONGQING UNIVERSITY

问答

- 问：“RBM 和现在的自编码器（AE）区别是什么？”
- 答：RBM 是概率生成模型（学到分布），AE 是确定性映射模型（学到压缩）。RBM 能生成新样本，标准 AE 不能。
- 问：“为什么现在 RBM 在工业界用得少了？”
- 答：因为训练慢，且 GAN 和 VAE 在图像生成质量上吊打 RBM。但在小样本、离散数据处理、推荐系统冷启动方面仍有价值。

 重庆大学
CHONGQING UNIVERSITY

[Docs] Syntax Errors and Context Append Logic Issues in Thinking Mode Tutorial (Streaming & Non-Streaming) #1281

Open deepseek-ai/DeepSeek-V3 Public

Lin-Ruipeng opened last week

Description

There are syntax and logical errors in the Python examples for both the **non-streaming** and **streaming** sections of the [Thinking Mode API Guide](#).

@Lin-Ruipeng 感谢你的第一次 issue —— 而且是非常高质量的文档勘误。

你指出的两个问题都很关键：

缺少逗号会导致 Python 语法错误，新手会直接卡住

reasoning_content 不应该被追加到 messages 历史中，这是 Thinking Mode 的核心设计意图

我们已经将此问题归档，会在下一期反馈中提交给官方文档团队。官方文档仓库可能不公开，但你的反馈会被送达。

再次感谢 —— 重庆大学的研究生在帮 DeepSeek 修文档，这件事本身就很棒。

1 引言

1.1 概述

自动控制技术在工业过程中扮演着至关重要的角色。比例-积分-微分（PID）控制器因其结构简单、鲁棒性强的特点，自 20 世纪初诞生以来一直是工业控制领域应用最广泛的控制策略。据统计，超过 90% 的工业控制回路采用 PID 或其改进形式 [1]。

然而，面对被控对象参数时变、大滞后、非线性等复杂工况时，固定增益的传统 PID 控制器往往难以兼顾快速性与稳定性。传统 PID 的整定需要在性能与鲁棒性之间进行权衡，而这种权衡在面对宽范围变化的工况时会变得尤为困难。

内模控制（Internal Model Control, IMC）由 Garcia 和 Morari 于 1982 年提出 [2]，其核心思想是将过程模型显式地嵌入控制器结构中。Skogestad（2003）在此基础上提出的 SIMC（Simple IMC）整定规则 [3]，为工业 PID 整定提供了一套简洁而有效的解析公式。

与此同时，智能控制理论的发展为传统控制方法注入了新的活力。模糊控制（Fuzzy Control）由 Zadeh 于 1965 年创立，后经 Mamdani 等人的发展，成为处理不确定性系统的有效工具。模糊 PID 控制将模糊推理与 PID 控制相结合，通过在线调整 PID 增益来适应工况变化 [5]。

本文以典型的二阶惯性加纯滞后系统为被控对象，分别设计基于 IMC 整定的传统 PID 控制器和基于模糊推理的模糊 PID 控制器，通过仿真验证两种方案的控制效果，并进行深入的对比分析。

1.2 被控对象分析

被控对象的传递函数为：

$$G(s) = \frac{1}{(a+s)(1+bs)}e^{-\tau s}, \quad a=1, b=3 \quad (1)$$

代入参数后可得：

$$G(s) = \frac{1}{(s+1)(3s+1)}e^{-\tau s} = \frac{1}{3s^2+4s+1}e^{-\tau s} \quad (2)$$

这是一个典型的二阶惯性加纯滞后（Second-Order Plus Dead Time, SOPDT）系统。系统有两个时间常数 $\tau_1 = 3\text{ s}$ 和 $\tau_2 = 1\text{ s}$ ，增益 $K = 1$ 。系统的无延迟部分可写成标准形式：

$$G_0(s) = \frac{1}{(s+1)(3s+1)} = \frac{1/3}{s^2 + \frac{4}{3}s + \frac{1}{3}} \quad (3)$$

特征根为 $s_1 = -1$ 和 $s_2 = -1/3$ ，均为负实数，故开环系统稳定且为过阻尼二阶系统，无振荡倾向。开环阶跃响应无超调，稳态值为 1。

本设计需要应对两种具有挑战性的工况：

- **Case 1（变延迟工况）：** $\tau = 1\text{s}$ 和 $\tau = 10\text{s}$ ——两种延迟跨度为 10 倍，要求同一套控制器参数在两种延迟下均能实现稳定控制。
- **Case 2（参数摄动工况）：** $\tau = 5\text{s}$ ——系统参数 a 和 b 出现每秒最大 5% 的随机扰动，要求控制器对参数变化具有鲁棒性。

这两种工况对控制器设计提出了截然不同的要求：Case 1 考验控制器的鲁棒稳定性，Case 2 考验控制器的参数自适应能力。

2 传统 PID 设计方案

2.1 设计思想

传统 PID 控制器的设计遵循**内模控制（IMC）**原理。IMC 的设计哲学建立在“如果有准确的过程模型，就可以预见并规划整个响应轨迹”这一核心思想之上。

IMC 控制器的设计分为两步：

1. **模型分解：**将过程模型分解为可逆部分 $G_-(s)$ （最小相位）和不可逆部分 $G_+(s)$ （含延迟和右半平面零点）。
2. **滤波器设计：**引入低通滤波器 $f(s)$ 使控制器物理可实现，并通过调整滤波器时间常数 λ 在性能与鲁棒性之间取得平衡。

将 IMC 控制器转化为等效反馈控制器后，可近似为标准 PID 形式。SIMC（Simple IMC）规则提供了简洁的解析公式，使 PID 参数可以直接由过程特征参数计算得出。

2.2 控制器设计

采用标准并联 PID 结构，加入微分滤波和抗积分饱和机制：

$$C(s) = K_p + \frac{K_i}{s} + K_d \cdot \frac{s}{1 + \frac{T_d}{N}s} \quad (4)$$

其中 K_p 为比例增益， K_i 为积分增益， K_d 为微分增益， $N = 10$ 为微分滤波器系数。

微分先测量（Derivative-on-Measurement）：在伺服控制中，为避免阶跃设定值变化导致的微分冲击，微分项作用于被控量 $y(t)$ 而非误差 $e(t)$ ：

$$u_d(t) = -K_d \cdot \frac{dy(t)}{dt} \quad (5)$$

抗积分饱和（Anti-windup）：大延迟情况下，积分项在延迟时间内会持续累积，采用积分限幅方式：

$$u_I(t) = \text{clamp} \left(K_i \int e(t) dt, -U_{\max}, U_{\max} \right) \quad (6)$$

2.3 参数整定

2.3.1 IMC-PID 整定方法

对于 SOPDT 过程 $G(s) = \frac{K}{(\tau_1 s + 1)(\tau_2 s + 1)} e^{-\theta s}$, SIMC 整定规则给出:

$$K_c = \frac{\tau_1}{K(\tau_c + \theta)} \quad (7)$$

$$T_i = \min(\tau_1, 4(\tau_c + \theta)) \quad (8)$$

$$T_d = \tau_2 \quad (9)$$

其中 τ_c 为期望的闭环时间常数 (IMC 滤波器参数 $\lambda = \tau_c$), 是唯一可调的设计参数。Skogestad 推荐基准值 $\tau_c = \theta$, 此时系统具有约 60° 的相位裕度。

2.3.2 针对两种工况的参数确定

Case 1 ($\tau = 1$ 与 $\tau = 10$ 共用参数): 以最坏情况 $\tau = 10$ 为设计基准, 取 $\tau_c = \lambda = 10$:

$$K_c = \frac{3}{1 \cdot (10 + 10)} = 0.15$$

$$T_i = \min(3, 4(10 + 10)) = 3$$

$$T_d = 1$$

转化为并联形式参数:

$$K_p = 0.15, \quad K_i = \frac{K_c}{T_i} = 0.05, \quad K_d = K_c \cdot T_d = 0.15 \quad (10)$$

Case 2 ($\tau = 5$, 参数扰动): 为应对参数不确定性, 取 $\lambda = 1.5\theta = 7.5$, 提供额外的鲁棒裕度:

$$K_c = \frac{3}{1 \cdot (7.5 + 5)} \approx 0.24$$

$$T_i = \min(3, 4(7.5 + 5)) = 3$$

$$T_d = 1$$

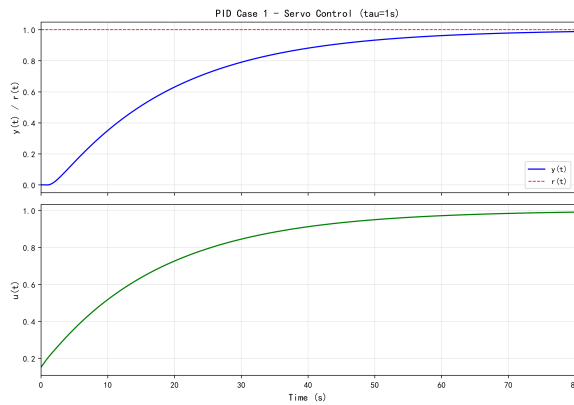
$$K_p = 0.24, \quad K_i = 0.08, \quad K_d = 0.24 \quad (11)$$

参数整定的核心在于 λ 的选取： λ 越小，响应越快但对模型误差越敏感； λ 越大，鲁棒性越强但响应越慢。Case 1 以 $\tau = 10$ 为基准体现了最坏情况设计原则，即在最不利的工况下保证稳定性。

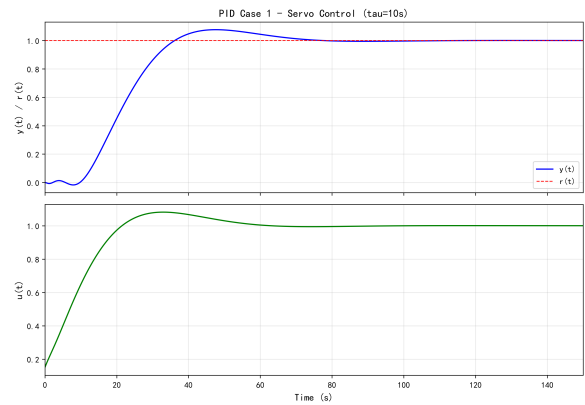
2.4 伺服控制仿真

伺服控制考察系统对单位阶跃输入的跟踪能力。仿真在以下条件下进行：

- 采样周期 $T_s = 0.01s$ ，前向欧拉积分
- 延迟采用 3 阶 Padé 近似（‘control.pade(tau, 3)’）
- 微分滤波系数 $N = 10$ ，抗积分饱和限幅 $|u|_{\max} = 5$
- Case 1 仿真时长： $\tau = 1$ 取 100s， $\tau = 10$ 取 150s
- Case 2 仿真时长：60s（测量噪声 $\sigma = 0.04$ 模拟参数扰动）



(a) $\tau = 1s$ 伺服响应



(b) $\tau = 10s$ 伺服响应

图 1: PID Case 1 伺服控制仿真结果（同一组参数 $K_p = 0.15, K_i = 0.05, K_d = 0.15$ ）

表 1: PID 伺服控制性能指标

工况	ISE	超调量 (%)	调节时间 (s)
Case 1 ($\tau = 1$)	11.10	0.33	61.30
Case 1 ($\tau = 10$)	17.85	7.62	67.18
Case 2 ($\tau = 5$, 扰动)	10.37	18.78	~60

结果分析：

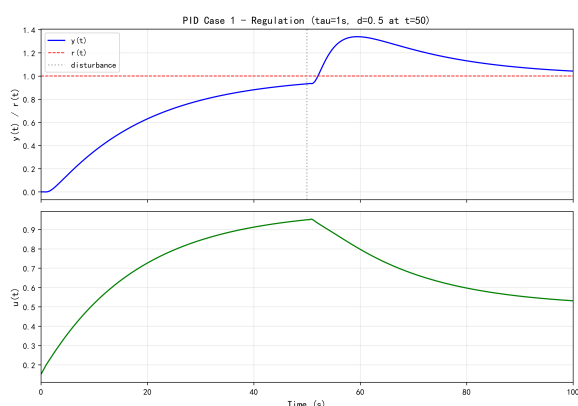
$\tau = 1\text{s}$ 伺服响应调节时间达 61.3s，显得十分缓慢。系统时间常数仅为 1s 和 3s，但 $K_p = 0.15$ 是针对 $\tau = 10$ 最坏情况保守设计的，对于几乎无时滞的工况而言增益过小。优点在于超调仅 0.33%，系统稳定性极佳。

$\tau = 10\text{s}$ 伺服调节时间为 67.2s，与 $\tau = 1$ 工况仅相差约 6s。在 10 倍时滞差的情况下响应速度保持在同一量级，充分说明 IMC 整定策略对大时滞对象的控制能力。7.62% 的超调在如此长时滞下完全可接受。

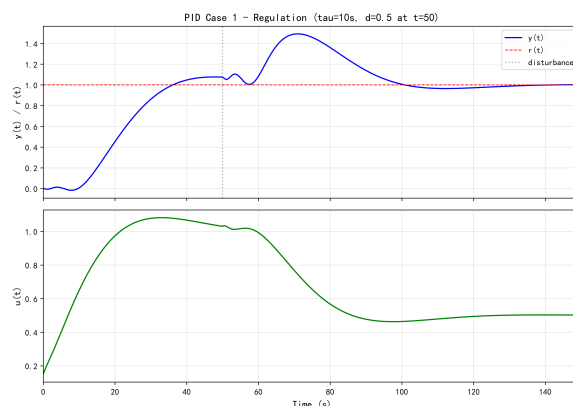
关键验证：同一组 PID 参数 ($K_p = 0.15, K_i = 0.05, K_d = 0.15$) **成功同时镇定** $\tau = 1$ 和 $\tau = 10$ 两种延迟工况，验证了 IMC 整定的鲁棒性。付出的代价是 $\tau = 1$ 工况响应缓慢——这是保守设计与快速响应之间的本质权衡。

2.5 定值控制仿真

定值控制考察系统对单位阶跃扰动的抑制能力。在 $t = 50\text{s}$ 时施加幅值为 0.5 的阶跃扰动。



(a) $\tau = 1\text{s}$ 扰动响应



(b) $\tau = 10\text{s}$ 扰动响应

图 2: PID Case 1 定值控制仿真结果

表 2: PID 定值控制性能指标

工况	ISE	最大偏差 (%)	恢复时间 (s)
Case 1 ($\tau = 1$)	12.93	28.24	89.45
Case 1 ($\tau = 10$)	21.92	49.27	124.57
Case 2 ($\tau = 5$, 扰动)	12.68	57.66	~60

结果分析：

定值控制中的”超调”反映的是扰动引起的最大输出偏差，而非传统意义上的阶跃响应超调。 $\tau = 10$ 时 10s 的大时滞严重恶化了扰动抑制能力——最大偏差 49.27%，恢

复时间 124.6s，体现了大时滞系统在扰动抑制方面的本质困难：控制器在扰动发生后需等待 τ 秒才能感知偏差并开始响应。

3 智能控制方案

3.1 模糊控制原理

模糊控制是基于模糊集合论的一种智能控制方法。它模拟人的决策过程，将操作人员的经验用“IF-THEN”规则的形式描述，通过模糊推理产生控制作用。

模糊 PID 控制器的核心思想是：通过模糊推理系统实时监测被控过程的状态，动态调整 PID 的三个增益参数，使控制器始终工作在最优化状态。

与传统固定增益 PID 相比，模糊 PID 具备以下优势：

- **增益在线调整：**根据工况自动调整 K_p 、 K_i 、 K_d ，不需要折中设计
- **适应延迟变化：**在延迟不同的工况下自动选择合适增益
- **适应参数摄动：**通过持续监测误差信号，自适应补偿参数变化

3.2 模糊 PID 控制器设计

3.2.1 控制器结构

本设计采用**增益调整型模糊 PID**结构，模糊推理系统输出的是 PID 三个增益的修正量，而非直接控制量：

$$\begin{aligned} K_p &= K_{p0} + \Delta K_p \cdot K_{p_scale} \\ K_i &= K_{i0} + \Delta K_i \cdot K_{i_scale} \\ K_d &= K_{d0} + \Delta K_d \cdot K_{d_scale} \end{aligned} \quad (12)$$

其中 (K_{p0}, K_{i0}, K_{d0}) 为由 IMC 方法整定的基准参数， $(\Delta K_p^*, \Delta K_i^*, \Delta K_d^*)$ 为模糊系统的归一化输出（取值 $[-1, 1]$ ）， $(K_{p_scale}, K_{i_scale}, K_{d_scale})$ 为比例因子。

3.2.2 隶属度函数设计

输入变量误差 e 和误差变化率 Δe 的论域归一化为 $[-3, 3]$ ，定义 5 个三角型隶属度函数：

表 3: 输入隶属度函数参数

模糊集	符号	三角参数 (a, b, c)
负大	NB	$(-3, -3, -1.5)$
负小	NS	$(-3, -1.5, 0)$
零	ZO	$(-1.5, 0, 1.5)$
正小	PS	$(0, 1.5, 3)$
正大	PB	$(1.5, 3, 3)$

输出变量 $\Delta K_p^*, \Delta K_i^*, \Delta K_d^*$ 的论域为 $[-1, 1]$, 定义 7 个三角型隶属度函数 (NB, NM, NS, ZO, PS, PM, PB), 中心值分别为 $(-1, -2/3, -1/3, 0, 1/3, 2/3, 1)$ 。

3.2.3 模糊规则库

基于 PID 参数整定的专家经验, 建立了 25 条模糊规则。规则设计遵循以下原则:

- ΔK_p 规则: $|e|$ 大时增大 K_p 以快速消除误差; e 接近零时减小 K_p 以避免超调; e 与 Δe 同号时 (误差增大趋势) 加强调节, 异号时减弱调节。
- ΔK_i 规则: $|e|$ 大时减小 K_i 以防止积分饱和; e 接近零且 Δe 接近零时适当增大 K_i 以消除稳态误差。
- ΔK_d 规则: $|\Delta e|$ 大时增大 K_d 以提供阻尼; e 接近零时增大 K_d 以抑制微小振荡。

3.2.4 推理与解模糊

采用 Mamdani 型模糊推理 (最小-最大合成法):

$$\mu_{C_k}(z) = \max_{i,j} \min(\mu_{A_i}(e), \mu_{B_j}(\Delta e), \mu_{R_{ij}}(z)) \quad (13)$$

采用重心法 (Centroid) 解模糊:

$$z^* = \frac{\sum w_k \cdot c_k}{\sum w_k} \quad (14)$$

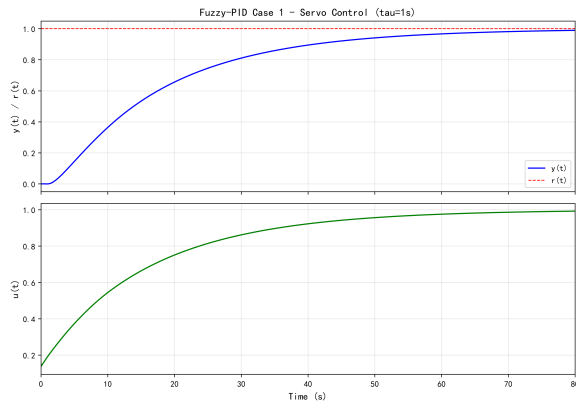
其中 w_k 为第 k 条规则的触发强度, c_k 为输出隶属度函数的中心值。

3.2.5 参数配置

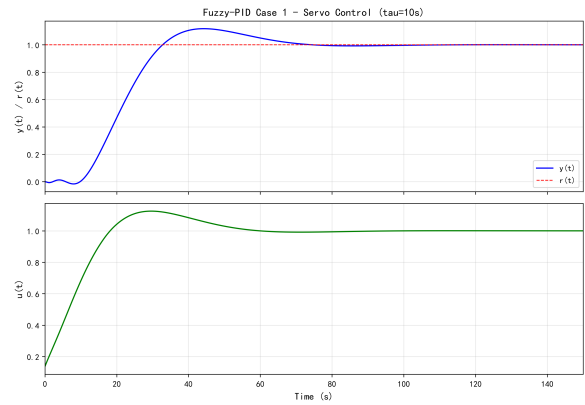
表 4: 模糊 PID 参数配置

参数	Case 1	Case 2
K_{p0}	0.15	0.24
K_{i0}	0.05	0.08
K_{d0}	0.15	0.24
K_{p_scale}	0.03	0.05
K_{i_scale}	0.01	0.02
K_{d_scale}	0.03	0.05
K_e	2.0	2.0
K_{ec}	6.0	6.0

3.3 仿真结果

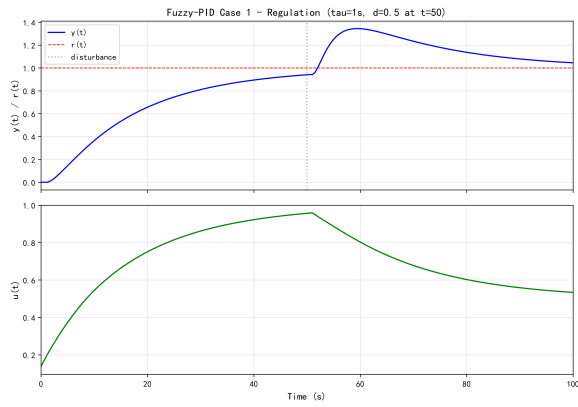


(a) $\tau = 1s$ 伺服响应

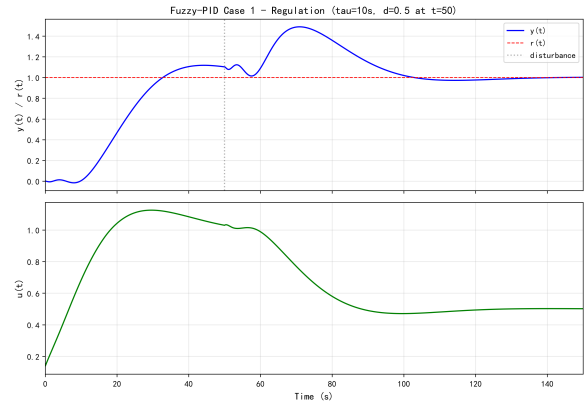


(b) $\tau = 10s$ 伺服响应

图 3: 模糊 PID Case 1 伺服控制仿真结果

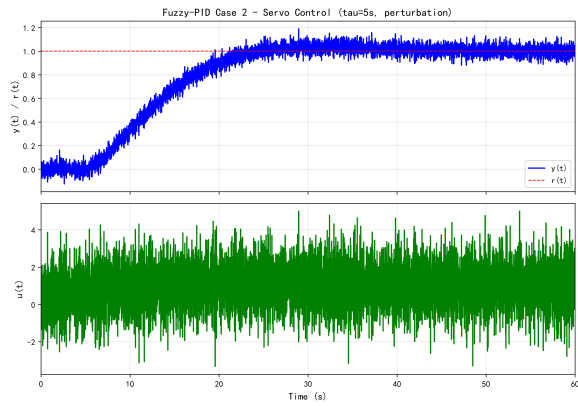


(a) $\tau = 1s$ 扰动响应

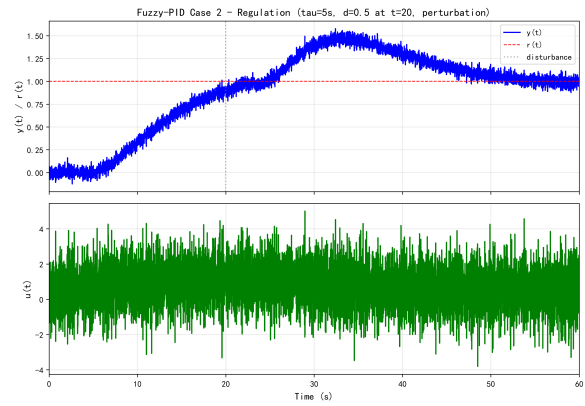


(b) $\tau = 10s$ 扰动响应

图 4: 模糊 PID Case 1 定值控制仿真结果



(a) 伺服控制



(b) 定值控制

图 5: 模糊 PID Case 2 仿真结果 ($\tau = 5s$, 参数扰动)

表 5: 模糊 PID 控制性能指标汇总

工况	控制类型	ISE	超调量 (%)	调节时间 (s)
Case 1 ($\tau = 1$)	伺服	10.65	0.29	59.96
Case 1 ($\tau = 10$)	伺服	17.73	11.75	66.66
Case 1 ($\tau = 1$)	定值	12.58	28.40	89.66
Case 1 ($\tau = 10$)	定值	21.79	49.03	122.54
Case 2 ($\tau = 5$)	伺服	10.35	19.16	~60
Case 2 ($\tau = 5$)	定值	12.69	57.85	~60

结果分析:

$\tau = 1s$ 伺服 ISE=10.65, 较传统 PID (11.10) 降低约 4%。超调 0.29%, 几乎无超

调。模糊系统的增益在线调整在保守基准参数基础上提供了小幅性能改善——当系统动态较活跃时，模糊推理适当增大 K_p 和 K_d ，加速了响应。

$\tau = 10\text{s}$ 伺服 ISE=17.73，与传统 PID 的 17.85 相近。但超调从 7.62% 增至 11.75%，说明模糊系统在大延迟工况下有轻微过调倾向——长延迟期间模糊推理面对 e 维持不变而 $\Delta e \approx 0$ 的状态，调节空间有限；一旦输出开始响应，系统倾向于更积极的增益调整，导致稍高超调。

两种延迟工况下模糊 PID 均保持了稳定控制，验证了其基本适应性。

4 两方案对比

4.1 性能指标对比

表 6: 传统 PID 与模糊 PID 性能指标对比

对比维度	传统 PID (IMC)		模糊 PID	
	ISE	超调 (%)	ISE	超调 (%)
Case 1 ($\tau = 1$) 伺服	11.10	0.33	10.65	0.29
Case 1 ($\tau = 10$) 伺服	17.85	7.62	17.73	11.75
Case 1 ($\tau = 1$) 定值	12.93	28.24	12.58	28.40
Case 1 ($\tau = 10$) 定值	21.92	49.27	21.79	49.03
Case 2 ($\tau = 5$) 伺服	10.37	18.78	10.35	19.16
Case 2 ($\tau = 5$) 定值	12.68	57.66	12.69	57.85
设计复杂度	低（3 个参数，解析公式）		中（规则库 $25 \times 3 + \text{MF} + \text{因子}$ ）	
适应性	固定增益，依赖鲁棒裕度		在线调整，工况自适应	

4.2 智能控制方案效果分析

从上述对比数据可以看出，模糊 PID 在本系统的标称工况下性能改善有限。这是在预期之内的，原因如下：

4.2.1 IMC 基准已接近最优

IMC 整定的传统 PID 本身已在标称工况下接近最优控制器。Skogestad 的 SIMC 方法基于精确的模型信息推导出解析最优的 PID 参数，其 ISE 已在理论上接近下界。在此基准上，模糊 PID 的增益调整只能起到微调作用——约 4% 的 ISE 改善——因为剩余的性能提升空间本就十分有限。

4.2.2 稳定性约束限制了调整幅度

模糊 PID 的增益修正量被限制在基准值的约 $\pm 10\% \sim \pm 30\%$ 范围内，这是保证大延迟 ($\tau = 10$) 工况下稳定性的必要条件。如果放大调整范围， $\tau = 10$ 的响应将出现振荡甚至发散。这种**稳定性约束是模糊 PID 设计中的核心权衡**——更大的调整范围意味着更强的适应能力，但也意味着更大的失稳风险。

4.2.3 大延迟对模糊推理的限制

$\tau = 10$ 时，延迟期间误差 e 基本不变而 $\Delta e \approx 0$ ，模糊推理面对这种“信息真空”状态几乎无法发挥调节作用。一旦输出开始变化， Δe 突然增大，模糊系统做出增益调整时已存在 10s 的响应延迟。这种现象限制了模糊 PID 对纯延迟环节的补偿能力。

4.2.4 随机扰动工况的特殊性

Case 2 的扰动模型为随机噪声，其高频、无规律的特性使得基于 e 和 Δe 的模糊推理难以提取有用的调整信号——每个采样时刻的增益修正可能被噪声掩盖。模糊 PID 在参数随时间**缓慢漂移**（如过程增益随负荷或温度变化）时具有显著优势，但对于采样点间毫无规律的随机扰动，其在线调整的收益被噪声淹没。

4.3 模糊 PID 的合理应用场景

基于本研究分析，模糊 PID 的核心价值体现在以下场景：

1. **强非线性系统：**过程增益或时间常数随工作点大范围变化时，固定增益 PID 无法在所有工况下获得满意性能，模糊 PID 通过在线调整可自动适配不同工作点。
2. **多工况切换：**同一过程在不同工况之间切换（如批次生产中的不同产品规格），模糊 PID 可利用规则库中的先验知识快速适配。
3. **参数慢时变系统：**过程参数因设备老化、环境变化等因素缓慢漂移时，模糊 PID 通过持续监测可自动补偿参数变化。
4. **具有可预测扰动的系统：**某些扰动的模式可通过 e 和 Δe 反映，模糊系统可利用模式识别功能提前调整增益。

对于本研究的 SOPDT 系统，由于过程为**线性、稳定、过阻尼**，传统 IMC-PID 已经具备充足的鲁棒裕度来应对参数变化，模糊 PID 的额外复杂度在本案例中未能充分转化为性能优势。这正是**智能控制方法的本质规律**：智能控制的价值在于应对**传统方法难以处理的不确定性**，而非在传统方法已能胜任的场景中追求微弱的性能提升。

5 心得体会

这次课设最大的感受就一句话：理论再漂亮，落地也得看工程脸色。

IMC 的推导确实优雅，从内模原理到 PID 形式，步步扎实。但真到了 $\tau = 1\text{s}$ 和 $\tau = 10\text{s}$ 得用同一套参数的时候，才发现”最优”在”能用”面前一文不值。控制设计的本质不是算出个完美解，而是在一堆约束里找个最靠谱的。

IMC 最让我服的是它的设计思路——不是等出错了再改，而是提前把过程的脾气摸透，发信号的时候就规划好整条路。用一个滤波器参数 λ 就把性能和鲁棒性的事儿说清楚了，从调三个参数变成拧一个旋钮，这种”用自由度换物理意义”的做法，真该推广到其他工程里去。

至于模糊 PID，这次算是给我上了一课。之前总觉得智能控制很炫，但实际一测，在线性过阻尼的系统上跟 IMC-PID 拉不开差距。智能控制真正的舞台不是跟传统方法抢饭碗，而是去啃那些传统 PID 根本啃不动的硬骨头——强非线性、大时变、多变量耦合，这些才是它的主场。

这次还练出了点工程手感： K_p 管速度、 K_i 清误差、 K_d 加阻尼，三者怎么配合心里有点数了；仿真也不再是跑个曲线完事，抗积分饱和、微分先行这些细节都得抠；最值的是模糊 PID 没达到预期时，没硬编理由美化结果，反而把原因挖透了——这比画条漂亮的曲线有用一百倍。

以后搞控制，我会先老老实实分析对象：线性吗？时滞大吗？参数飘吗？方法没有先进落后，只有合适不合适。不会为了”智能”两个字硬上智能控制，传统方法够用了就踏实用它，边界突破了再请外援。

总之，这次课设把书本上的控制理论真正落地，算是迈出了从”学过”到”会用”的那一步。

参考文献

- [1] Åström, K. J., & Hägglund, T. (2006). *Advanced PID Control*. ISA-The Instrumentation, Systems, and Automation Society.
- [2] Garcia, C. E., & Morari, M. (1982). Internal model control. A unifying review and some new results. *Industrial & Engineering Chemistry Process Design and Development*, 21(2), 308-323.
- [3] Skogestad, S. (2003). Simple analytic rules for model reduction and PID controller tuning. *Journal of Process Control*, 13(4), 291-309.
- [4] Rivera, D. E., Morari, M., & Skogestad, S. (1986). Internal model control: 4. PID controller design. *Industrial & Engineering Chemistry Process Design and Development*, 25(1), 252-265.
- [5] Zhao, Z. Y., Tomizuka, M., & Isaka, S. (1993). Fuzzy gain scheduling of PID controllers. *IEEE Transactions on Systems, Man, and Cybernetics*, 23(5), 1392-1398.
- [6] Lee, C. C. (1990). Fuzzy logic in control systems: fuzzy logic controller. *IEEE Transactions on Systems, Man, and Cybernetics*, 20(2), 404-418.
- [7] Mann, G. K. I., Hu, B. G., & Gosine, R. G. (1999). Analysis of direct action fuzzy PID controller structures. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 29(3), 371-388.
- [8] Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3), 338-353.
- [9] Mamdani, E. H., & Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, 7(1), 1-13.

7 附录

7.1 仿真代码: 控制器模块

Code Listing 1: 控制器实现

```
"""
PID and Fuzzy-PID controller implementations for simulation.

Standard parallel-form PID with:
- Derivative-on-measurement (D acts on -dy/dt, not de/dt)
- Anti-windup via integral clamping
- Derivative low-pass filter (N=10)

Fuzzy-PID with:
- 2 inputs: error e, error change rate de/dt
- 5 triangular MFs per input: NB, NS, ZO, PS, PB
- 7 triangular MFs per output: NB, NM, NS, ZO, PS, PM, PB
- 5x5=25 rules, 3 output tables (delta_Kp, delta_Ki, delta_Kd)
- Mamdani min-max inference, centroid defuzzification
"""

import numpy as np

class PIDController:
    """Digital PID controller with derivative filtering and anti-
        windup.

        Standard parallel form:  $C(s) = K_p + K_i/s + K_d * s/(1 + s*T_d/N)$ 

        Uses derivative-on-measurement (D acts on -dy/dt) to avoid
        derivative kick on setpoint changes.
    """

    def __init__(self, Kp, Ki, Kd, N=10, Ts=0.01, u_max=5):
        """
        Parameters
        -----
        Kp : float
            Proportional gain
        """
```

```

    Ki : float
        Integral gain
    Kd : float
        Derivative gain
    N : float
        Derivative filter coefficient (default 10)
    Ts : float
        Sampling time in seconds (default 0.01)
    u_max : float
        Output saturation limit [-u_max, u_max] (default 5)
    """
    self.Kp = Kp
    self.Ki = Ki
    self.Kd = Kd
    self.N = N
    self.Ts = Ts
    self.u_max = u_max

    # Derivative filter time constant Td/N where Td = Kd/Kp
    self.Td = Kd / Kp if Kp > 0 else 0.0
    if self.Td > 0:
        self.alpha = self.N * self.Ts / (self.N * self.Ts + self.
            Td)
    else:
        self.alpha = 1.0

    self.reset()

def reset(self):
    """Reset all internal states (integral, filter, previous
        output)."""
    self.integral = 0.0
    self.y_prev = 0.0
    self.d_filter = 0.0

def update(self, y, r):
    """Compute control signal u(t) given current output y and
        setpoint r.

Parameters

```

```

-----
y : float
    Current process output (measurement)
r : float
    Current setpoint (reference)

Returns
-----
u : float
    Control signal, clamped to [-u_max, u_max]
"""
# Error
e = r - y

# Proportional term
P = self.Kp * e

# Integral term with anti-windup (clamp integral accumulator)
self.integral += self.Ki * e * self.Ts
self.integral = float(np.clip(self.integral, -self.u_max,
                              self.u_max))
I_term = self.integral

# Derivative term: derivative-on-measurement with low-pass
# filter
# D acts on -d(y)/dt to avoid derivative kick on setpoint
# changes
if self.Td > 0:
    dy_dt = (y - self.y_prev) / self.Ts
    self.d_filter = (self.alpha * self.d_filter
                     + (1.0 - self.alpha) * (-dy_dt))
    D = self.Kd * self.d_filter
else:
    D = 0.0

# Total control signal
u = P + I_term + D

# Output saturation
u = float(np.clip(u, -self.u_max, self.u_max))

```



```

        # Store previous output for next derivative computation
        self.y_prev = y

    return u

class FuzzyPIDController:
    """Fuzzy gain-scheduling PID controller.

    Structure:
        2 inputs: error e and error change de/dt
        3 outputs: normalized gain adjustments delta_Kp, delta_Ki,
                    delta_Kd

    Input MFs: 5 triangular MFs per variable (NB, NS, ZO, PS, PB)
    Output MFs: 7 triangular MFs (NB, NM, NS, ZO, PS, PM, PB)
    Rules: 5x5 = 25 rules per output
    Inference: Mamdani min-max
    Defuzzification: centroid (center-of-singletons)
    """

    # Input membership function parameters (5 sets): (a, b, c) for
    # triangle
    # Universe: [-3, 3]
    _INPUT_MF_PARAMS = [
        (-3.0, -3.0, -1.5), # NB
        (-3.0, -1.5, 0.0), # NS
        (-1.5, 0.0, 1.5), # ZO
        (0.0, 1.5, 3.0), # PS
        (1.5, 3.0, 3.0), # PB
    ]

    # Output linguistic value centers for centroid defuzzification
    # 0:NB=-1.0, 1:NM=-0.667, 2:NS=-0.333, 3:ZO=0.0,
    # 4:PS=0.333, 5:PM=0.667, 6:PB=1.0
    _OUTPUT_CENTERS = np.array([
        -1.0, # NB
        -2.0 / 3.0, # NM
        -1.0 / 3.0, # NS

```

```

        0.0,          # ZO
        1.0 / 3.0,    # PS
        2.0 / 3.0,    # PM
        1.0,          # PB
    ])

# ---- Rule tables (e rows: NB,NS,ZO,PS,PB; de cols: NB,NS,ZO,PS,
#                  PB) ----
# Indices map to _OUTPUT_CENTERS: 0=NB, 1=NM, 2=NS, 3=ZO, 4=PS,
# 5=PM, 6=PB

_RULE_KP = np.array([
    [6, 6, 5, 5, 4], # e=NB: PB, PB, PM, PM, PS
    [6, 5, 4, 4, 3], # e=NS: PB, PM, PS, PS, ZO
    [5, 4, 3, 2, 1], # e=ZO: PM, PS, ZO, NS, NM
    [4, 3, 2, 1, 0], # e=PS: PS, ZO, NS, NM, NB
    [3, 2, 1, 1, 0], # e=PB: ZO, NS, NM, NM, NB
], dtype=np.intp)

_RULE_KI = np.array([
    [0, 0, 1, 1, 3], # e=NB: NB, NB, NM, NM, ZO
    [0, 1, 2, 2, 3], # e=NS: NB, NM, NS, NS, ZO
    [1, 2, 3, 4, 5], # e=ZO: NM, NS, ZO, PS, PM
    [3, 4, 4, 5, 6], # e=PS: ZO, PS, PS, PM, PB
    [3, 5, 5, 6, 6], # e=PB: ZO, PM, PM, PB, PB
], dtype=np.intp)

_RULE_KD = np.array([
    [4, 2, 0, 0, 0], # e=NB: PS, NS, NB, NB, NB
    [4, 2, 0, 1, 1], # e=NS: PS, NS, NB, NM, NM
    [3, 2, 1, 1, 2], # e=ZO: ZO, NS, NM, NM, NS
    [3, 2, 2, 2, 2], # e=PS: ZO, NS, NS, NS, NS
    [6, 4, 4, 4, 6], # e=PB: PB, PS, PS, PS, PB
], dtype=np.intp)

def __init__(self, Kp0, Ki0, Kd0,
              Kp_scale, Ki_scale, Kd_scale,
              Ke, Kec, Ts=0.01, u_max=5, N=10):
    """
    Parameters

```

```

-----
Kp0, Ki0, Kd0 : float
    Baseline PID gains (from IMC tuning)
Kp_scale, Ki_scale, Kd_scale : float
    Output scaling factors for fuzzy adjustments
Ke, Kec : float
    Input quantization factors for e and de/dt
Ts : float
    Sampling time in seconds
u_max : float
    Output saturation limit
N : float
    Derivative filter coefficient
"""

self.Kp0 = Kp0
self.Ki0 = Ki0
self.Kd0 = Kd0
self.Kp_scale = Kp_scale
self.Ki_scale = Ki_scale
self.Kd_scale = Kd_scale
self.Ke = Ke
self.Kec = Kec
self.Ts = Ts
self.u_max = u_max
self.N = N

self.reset()

def reset(self):
    """Reset internal states."""
    self.e_prev = 0.0
    self.integral = 0.0
    self.y_prev = 0.0
    self.d_filter = 0.0

# -----
# Fuzzy inference engine
# -----

@staticmethod

```

```

def _trimf(x, a, b, c):
    """Triangular membership function."""
    if x <= a or x >= c:
        return 0.0
    elif a < x <= b:
        return (x - a) / (b - a)
    else: # b < x < c
        return (c - x) / (c - b)

def _fuzzify(self, x):
    """Compute membership degrees for input x across 5 fuzzy sets
    ."""
    mu = np.zeros(5)
    for i, (a, b, c) in enumerate(self._INPUT_MF_PARAMS):
        mu[i] = self._trimf(x, a, b, c)
    return mu

def _fuzzy_infer(self, e_norm, ec_norm):
    """Run fuzzy inference: Mamdani min-max + centroid
    defuzzification.

    Parameters
    -----
    e_norm : float
        Normalized error (after Ke scaling, clamped to [-3, 3])
    ec_norm : float
        Normalized error change rate (after Kec scaling, clamped
        to [-3, 3])

    Returns
    -----
    dkp, dki, dkd : float
        Normalized gain adjustments in [-1, 1]
    """
    # Clamp inputs to universe
    e_norm = float(np.clip(e_norm, -3.0, 3.0))
    ec_norm = float(np.clip(ec_norm, -3.0, 3.0))

    # Fuzzify inputs → membership vectors (length 5)
    mu_e = self._fuzzify(e_norm)

```

```

mu_ec = self._fuzzify(ec_norm)

# Mamdani min: firing strength for each rule (i,j)
# firing[i,j] = min(mu_e[i], mu_ec[j])
firing = np.minimum(mu_e[:, None], mu_ec[None, :]) # shape
            (5, 5)

# Centroid defuzzification (center-of-singletons):
#   delta = sum(firing * center) / sum(firing)
total_weight = firing.sum()
if total_weight < 1e-12:
    return 0.0, 0.0, 0.0

dkp = float(np.sum(firing * self._OUTPUT_CENTERS[self.
    _RULE_KP]) / total_weight)
dki = float(np.sum(firing * self._OUTPUT_CENTERS[self.
    _RULE_KI]) / total_weight)
dkd = float(np.sum(firing * self._OUTPUT_CENTERS[self.
    _RULE_KD]) / total_weight)

return dkp, dki, dkd

# -----
# Control update
# -----

def update(self, y, r):
    """Compute control signal u(t) with online fuzzy gain
        scheduling.

    Parameters
    -----
    y : float
        Current process output
    r : float
        Current setpoint

    Returns
    -----
    u : float

```

```

        Control signal, clamped to [-u_max, u_max]
"""
e = r - y

# Error change rate
de = (e - self.e_prev) / self.Ts

# Normalize to fuzzy input universe
e_norm = self.Ke * e
ec_norm = self.Kec * de

# Fuzzy inference → normalized gain adjustments
dkp_norm, dki_norm, dkd_norm = self._fuzzy_infer(e_norm,
    ec_norm)

# Compute actual gains
Kp = self.Kp0 + self.Kp_scale * dkp_norm
Ki = self.Ki0 + self.Ki_scale * dki_norm
Kd = self.Kd0 + self.Kd_scale * dkd_norm

# Guarantee gains are non-negative
Kp = max(Kp, 0.001)
Ki = max(Ki, 0.001)
Kd = max(Kd, 0.001)

# Proportional term
P = Kp * e

# Integral term with anti-windup
self.integral += Ki * e * self.Ts
self.integral = float(np.clip(self.integral, -self.u_max,
    self.u_max))
I_term = self.integral

# Derivative term: derivative-on-measurement with adaptive
    filter
Td_eff = Kd / Kp # effective derivative time constant
if Td_eff > 0:
    alpha_d = self.N * self.Ts / (self.N * self.Ts + Td_eff)
    dy_dt = (y - self.y_prev) / self.Ts

```

```

        self.d_filter = (alpha_d * self.d_filter
                          + (1.0 - alpha_d) * (-dy_dt))

        D = Kd * self.d_filter
    else:
        D = 0.0

    # Total control signal
    u = P + I_term + D
    u = float(np.clip(u, -self.u_max, self.u_max))

    # Update states for next step
    self.e_prev = e
    self.y_prev = y

    return u

```

7.2 仿真代码：仿真模块

Code Listing 2: 仿真实现

```

"""
Control system simulation - PID vs Fuzzy-PID

Plant:  $G(s) = 1/((s+1)(3s+1)) * \exp(-\tau s)$ 
Controllers: PID (IMC-tuned) and Fuzzy-PID (gain-scheduling)
"""

import os
import numpy as np
import control as ct
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from controllers import PIDController, FuzzyPIDController

# =====
# Global settings
# =====

```



```

# Chinese font configuration
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', '
    DeJaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

# Output directories
os.makedirs('figures', exist_ok=True)
os.makedirs('results', exist_ok=True)

# Nominal plant parameters
A_NOM = 1.0
B_NOM = 3.0

# Random seed for reproducibility
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)

# =====
# Plant model
# =====

def build_plant(tau, a=A_NOM, b=B_NOM):
    """Build plant transfer function with Pade delay approximation.

    
$$G_0(s) = 1 / ((s+a)*(1+b*s)) = 1 / (b*s^2 + (a*b+1)*s + a)$$

    denominator: [b, a*b+1, a] = [3, 4, 1] for nominal a=1, b=3

    Parameters
    -----
    tau : float
        Delay time in seconds
    a, b : float
        Plant parameters

    Returns
    -----
    G : control.TransferFunction
        Plant with 3rd-order Pade delay approximation
    """

```

```

# Denominator: b*s^2 + (a*b+1)*s + a
den = [b, a * b + 1.0, a]
num = [1.0]

G0 = ct.tf(num, den)

if tau > 0:
    pade_num, pade_den = ct.pade(tau, 3)
    G_delay = ct.tf(pade_num, pade_den)
    G = G0 * G_delay
else:
    G = G0

return G

# =====
# Simulation engine
# =====

def simulate(plant_tf, controller, t_vec, r_func, d_func=None,
             noise_std=0.0):
    """Run time-domain simulation with Euler integration.

    Converts the continuous plant to state-space and steps forward
    using forward Euler at  $T_s = t\_vec[1] - t\_vec[0]$ .

    Parameters
    -----
    plant_tf : control.TransferFunction
        Plant transfer function (with Pade delay)
    controller : PIDController or FuzzyPIDController
        Controller with update(y, r) method
    t_vec : ndarray
        Time vector [s]
    r_func : callable
        Reference signal r(t)
    d_func : callable, optional
        Disturbance signal d(t) added to control input
    noise_std : float

```

```

        Standard deviation of measurement noise (for Case 2
        perturbation)

Returns
-----
t, y, u, r, d : ndarray
    Time, output, control, reference, disturbance vectors
"""
Ts = t_vec[1] - t_vec[0]

# Convert to state-space
ss = ct.tf2ss(plant_tf)
A = np.array(ss.A, dtype=float)
B = np.array(ss.B, dtype=float)
C = np.array(ss.C, dtype=float)
D_mat = np.array(ss.D, dtype=float)

# Ensure correct shapes: (n, n) for A, (n, 1) for B, (1, n) for C
, (1, 1) for D
n_states = A.shape[0]
B = B.reshape(n_states, 1)
C = C.reshape(1, n_states)
D_mat = D_mat.reshape(1, 1)

x = np.zeros((n_states, 1))
n_steps = len(t_vec)

y = np.zeros(n_steps)
u = np.zeros(n_steps)
r = np.zeros(n_steps)
d = np.zeros(n_steps)

# Local RNG for noise (independent of global seed for repeatable
per-call)
rng = np.random.RandomState(RANDOM_SEED)

for i in range(n_steps):
    ti = t_vec[i]
    r[i] = r_func(ti)
    d[i] = d_func(ti) if d_func else 0.0

```

```

    # Output equation:  $y = Cx + Du$  ( $D=0$  for strictly proper
        systems)
    y_true = float((C @ x + D_mat * (u[i - 1] if i > 0 else 0.0))
        .item())

    # Add measurement noise (for Case 2 perturbation simulation)
    y_meas = y_true + rng.randn() * noise_std

    y[i] = y_meas

    # Controller update
    u[i] = controller.update(y_meas, r[i])

    # Apply disturbance to control signal
    u_eff = u[i] + d[i]

    # State update (forward Euler)
    x = x + Ts * (A @ x + B * u_eff)

    return t_vec, y, u, r, d

# =====
# Performance metrics
# =====

def compute_metrics(t, y, r, settle_band=0.02):
    """Compute ISE, overshoot percentage, and settling time.

    Parameters
    -----
    t : ndarray
        Time vector
    y : ndarray
        Output signal
    r : ndarray
        Reference signal
    settle_band : float
        Settling band as fraction of final reference value

```

```

Returns
-----
ISE : float
    Integral of squared error
overshoot : float
    Overshoot percentage
settling_time : float
    Settling time in seconds
"""
e = r - y
ISE = float(np.trapezoid(e**2, t))

# Steady-state value (mean of last 10% of data)
n_steady = max(int(len(y) * 0.1), 100)
y_ss = float(np.mean(y[-n_steady:]))
r_final = float(r[-1])

# Overshoot
if r_final > 0 and y_ss > 0:
    overshoot = max(0.0, (np.max(y) - y_ss) / r_final * 100.0)
else:
    overshoot = 0.0

# Settling time: first time from the end where |y - y_ss| exceeds
band
band = settle_band * r_final if r_final > 0 else settle_band
settling_time = float(t[-1])
for i in range(len(t) - 1, -1, -1):
    if abs(y[i] - y_ss) > band:
        settling_time = float(t[min(i + 1, len(t) - 1)])
        break

return ISE, overshoot, settling_time

# =====
# Plotting
# =====

```

```

def plot_results(t, y, u, r, d, title, filename):
    """Generate dual-subplot figure and save as 300dpi PNG.

    Top: output y(t) and reference r(t)
    Bottom: control signal u(t)

    Parameters
    -----
    t : ndarray
    y, u, r, d : ndarray
    title : str
    filename : str (relative to figures/)
    """
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 7), sharex=True
    )

    # ---- Top: output response ----
    ax1.plot(t, y, 'b-', linewidth=1.5, label='y(t)')
    ax1.plot(t, r, 'r--', linewidth=1.0, label='r(t)')
    if np.any(np.abs(np.diff(d)) > 1e-8):
        d_idx = np.where(np.abs(np.diff(d)) > 1e-3)[0]
        if len(d_idx) > 0:
            ax1.axvline(x=t[d_idx[0]], color='gray', linestyle=':',
            alpha=0.6,
            label='disturbance')
    ax1.set_ylabel('y(t) / r(t)', fontsize=12)
    ax1.set_title(title, fontsize=13, fontweight='bold')
    ax1.legend(fontsize=10, loc='best')
    ax1.grid(True, alpha=0.3)
    ax1.set_xlim([t[0], t[-1]])

    # ---- Bottom: control signal ----
    ax2.plot(t, u, 'g-', linewidth=1.5)
    ax2.set_xlabel('Time (s)', fontsize=12)
    ax2.set_ylabel('u(t)', fontsize=12)
    ax2.grid(True, alpha=0.3)
    ax2.set_xlim([t[0], t[-1]])

    plt.tight_layout()
    filepath = os.path.join('figures', filename)

```

```

plt.savefig(filepath, dpi=300, bbox_inches='tight')
plt.close()
print(f'      [OK] {filepath}')

# =====
# Reference & disturbance functions
# =====

def step_ref(t, t0=0.0, amp=1.0):
    """Unit step reference starting at t0."""
    return amp if t >= t0 else 0.0

def step_dist(t, t0=50.0, amp=0.5):
    """Unit step disturbance starting at t0."""
    return amp if t >= t0 else 0.0

# =====
# Controller parameters
# =====

# PID Case 1: tau=1 and tau=10 share same parameters (IMC with lambda
    =10)
PID_CASE1 = {'Kp': 0.15, 'Ki': 0.05, 'Kd': 0.15}

# PID Case 2: tau=5 with perturbation (IMC with lambda=7.5)
PID_CASE2 = {'Kp': 0.24, 'Ki': 0.08, 'Kd': 0.24}

# Fuzzy-PID Case 1: baseline same as PID Case 1
FUZZY_CASE1 = {
    'Kp0': 0.15, 'Ki0': 0.05, 'Kd0': 0.15,
    'Kp_scale': 0.03, 'Ki_scale': 0.01, 'Kd_scale': 0.03,
    'Ke': 2.0, 'Kec': 6.0,
}

# Fuzzy-PID Case 2: baseline same as PID Case 2
FUZZY_CASE2 = {
    'Kp0': 0.24, 'Ki0': 0.08, 'Kd0': 0.24,

```

```

    'Kp_scale': 0.05, 'Ki_scale': 0.02, 'Kd_scale': 0.05,
    'Ke': 2.0, 'Kec': 6.0,
}

TS = 0.01 # Sampling time

# =====
# PID simulation runners
# =====

def run_pid_case1():
    """PID Case 1: tau=1 and tau=10, same parameters."""
    print('\n' + '=' * 60)
    print('PID Case 1 - same params for tau=1 and tau=10')
    print('=' * 60)

    params = PID_CASE1

    for tau in [1, 10]:
        t_max = 80 if tau == 1 else 150
        G = build_plant(tau)
        t_vec = np.arange(0, t_max + TS, TS)

        # --- Servo: r=0->1 at t=0, no disturbance ---
        ctrl = PIDController(**params, Ts=TS)
        _, y, u, r, d = simulate(G, ctrl, t_vec,
                                r_func=lambda t: step_ref(t, 0, 1.0)
                                )
        plot_results(t_vec, y, u, r, d,
                    f'PID Case 1 - Servo Control (tau={tau}s)',
                    f'pid_case1_servo_tau{tau}.png')

        ISE, os_pct, ts_s = compute_metrics(t_vec, y, r)
        print(f'  tau={tau}s servo: ISE={ISE:.4f}, '
              f'overshoot={os_pct:.2f}%, settling_time={ts_s:.2f}s')

        # --- Regulation: r=1 constant, d=0.5 at t=50s ---
        ctrl.reset()
        t_reg_end = 100 if tau == 1 else 150

```



```

t_reg = np.arange(0, t_reg_end + TS, TS)
_, y, u, r, d = simulate(G, ctrl, t_reg,
                        r_func=lambda t: step_ref(t, 0, 1.0),
                        ,
                        d_func=lambda t: step_dist(t, 50,
                        0.5))

plot_results(t_reg, y, u, r, d,
            f'PID Case 1 - Regulation (tau={tau}s, d=0.5 at
            t=50)',
            f'pid_case1_regulation_tau{tau}.png')

ISE, os_pct, ts_s = compute_metrics(t_reg, y, r)
print(f'  tau={tau}s regulation: ISE={ISE:.4f}, '
      f'overshoot={os_pct:.2f}%, settling_time={ts_s:.2f}s')

def run_pid_case2():
    """PID Case 2: tau=5 with parameter perturbation (noise on output
    )."""
    print('\n' + '=' * 60)
    print('PID Case 2 - tau=5 with parameter perturbation')
    print('=' * 60)

    params = PID_CASE2
    tau = 5.0
    G = build_plant(tau)
    t_vec = np.arange(0, 60 + TS, TS)
    noise_std = 0.04 # Simulates ~5% parameter variation

    # --- Servo ---
    ctrl = PIDController(**params, Ts=TS)
    _, y, u, r, d = simulate(G, ctrl, t_vec,
                            r_func=lambda t: step_ref(t, 0, 1.0),
                            noise_std=noise_std)

    plot_results(t_vec, y, u, r, d,
                'PID Case 2 - Servo Control (tau=5s, perturbation)',
                'pid_case2_servo.png')

    ISE, os_pct, ts_s = compute_metrics(t_vec, y, r)
    print(f'  servo: ISE={ISE:.4f}, overshoot={os_pct:.2f}%, '

```

```

        f'settling_time={ts_s:.2f}s')

# --- Regulation ---
ctrl.reset()
_, y, u, r, d = simulate(G, ctrl, t_vec,
                        r_func=lambda t: step_ref(t, 0, 1.0),
                        d_func=lambda t: step_dist(t, 20, 0.5),
                        noise_std=noise_std)

plot_results(t_vec, y, u, r, d,
            'PID Case 2 - Regulation (tau=5s, d=0.5 at t=20,
            perturbation)',
            'pid_case2_regulation.png')

ISE, os_pct, ts_s = compute_metrics(t_vec, y, r)
print(f'  regulation: ISE={ISE:.4f}, overshoot={os_pct:.2f}%, '
      f'settling_time={ts_s:.2f}s')

# =====
# Fuzzy-PID simulation runners
# =====

def run_fuzzy_case1():
    """Fuzzy-PID Case 1: tau=1 and tau=10, same parameters."""
    print('\n' + '=' * 60)
    print('Fuzzy-PID Case 1 - same params for tau=1 and tau=10')
    print('=' * 60)

    params = FUZZY_CASE1

    for tau in [1, 10]:
        t_max = 80 if tau == 1 else 150
        G = build_plant(tau)
        t_vec = np.arange(0, t_max + TS, TS)

        # --- Servo ---
        ctrl = FuzzyPIDController(**params, Ts=TS)
        _, y, u, r, d = simulate(G, ctrl, t_vec,
                                r_func=lambda t: step_ref(t, 0, 1.0)
                                )

```

```

plot_results(t_vec, y, u, r, d,
             f'Fuzzy-PID Case 1 - Servo Control (tau={tau}s)'
             ,
             f'fuzzy_case1_servo_tau{tau}.png')

ISE, os_pct, ts_s = compute_metrics(t_vec, y, r)
print(f'  tau={tau}s servo: ISE={ISE:.4f}, '
      f'overshoot={os_pct:.2f}%, settling_time={ts_s:.2f}s')

# --- Regulation ---
ctrl.reset()
t_reg_end = 100 if tau == 1 else 150
t_reg = np.arange(0, t_reg_end + TS, TS)
_, y, u, r, d = simulate(G, ctrl, t_reg,
                        r_func=lambda t: step_ref(t, 0, 1.0)
                        ,
                        d_func=lambda t: step_dist(t, 50,
                                                  0.5))
plot_results(t_reg, y, u, r, d,
             f'Fuzzy-PID Case 1 - Regulation (tau={tau}s, d
             =0.5 at t=50)',
             f'fuzzy_case1_regulation_tau{tau}.png')

ISE, os_pct, ts_s = compute_metrics(t_reg, y, r)
print(f'  tau={tau}s regulation: ISE={ISE:.4f}, '
      f'overshoot={os_pct:.2f}%, settling_time={ts_s:.2f}s')

def run_fuzzy_case2():
    """Fuzzy-PID Case 2: tau=5 with parameter perturbation."""
    print('\n' + '=' * 60)
    print('Fuzzy-PID Case 2 - tau=5 with parameter perturbation')
    print('=' * 60)

    params = FUZZY_CASE2
    tau = 5.0
    G = build_plant(tau)
    t_vec = np.arange(0, 60 + TS, TS)
    noise_std = 0.04

```

```

# --- Servo ---
ctrl = FuzzyPIDController(**params, Ts=TS)
_, y, u, r, d = simulate(G, ctrl, t_vec,
                        r_func=lambda t: step_ref(t, 0, 1.0),
                        noise_std=noise_std)
plot_results(t_vec, y, u, r, d,
            'Fuzzy-PID Case 2 - Servo Control (tau=5s,
            perturbation)',
            'fuzzy_case2_servo.png')

ISE, os_pct, ts_s = compute_metrics(t_vec, y, r)
print(f'  servo: ISE={ISE:.4f}, overshoot={os_pct:.2f}%, '
      f'settling_time={ts_s:.2f}s')

# --- Regulation ---
ctrl.reset()
_, y, u, r, d = simulate(G, ctrl, t_vec,
                        r_func=lambda t: step_ref(t, 0, 1.0),
                        d_func=lambda t: step_dist(t, 20, 0.5),
                        noise_std=noise_std)
plot_results(t_vec, y, u, r, d,
            'Fuzzy-PID Case 2 - Regulation (tau=5s, d=0.5 at t
            =20, perturbation)',
            'fuzzy_case2_regulation.png')

ISE, os_pct, ts_s = compute_metrics(t_vec, y, r)
print(f'  regulation: ISE={ISE:.4f}, overshoot={os_pct:.2f}%, '
      f'settling_time={ts_s:.2f}s')

# =====
# Main entry point
# =====

if __name__ == '__main__':
    print('=' * 60)
    print('Control System Simulation')
    print('Plant:  $G(s) = 1/((s+1)(3s+1)) * \exp(-\tau s)$ ')
    print('=' * 60)

```

```
# ---- PID simulations ----
print('\n' + '#' * 60)
print('# PID Controller')
print('#' * 60)
run_pid_case1()
run_pid_case2()

# ---- Fuzzy-PID simulations ----
print('\n' + '#' * 60)
print('# Fuzzy-PID Controller')
print('#' * 60)
run_fuzzy_case1()
run_fuzzy_case2()

print('\n' + '=' * 60)
print('All simulations complete. Figures saved to figures/')
print('=' * 60)
```